

# CSE 331

# AUTOMATA AND COMPUTABILITY

---

Building DFA

Spring 2026

# Building DFA



## Typical questions

Define a DFA which recognizes the given language  $L$ .

*or*

Prove that the (given) language  $L$  is regular.

# Building DFA

## Example

Define a DFA which recognizes

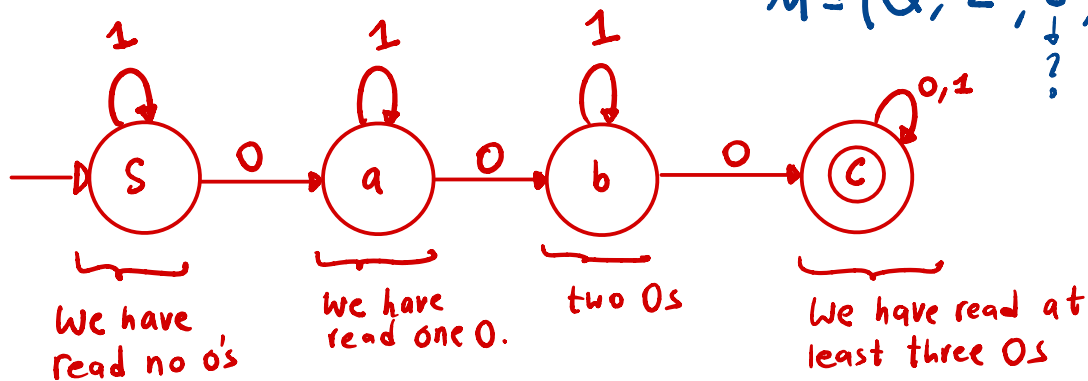
$L_1 = \{w \mid \text{all strings containing at least three 0s.}\}$

$L_1 = \{000, 01010, 011011001, \dots\}$

$M = (Q, \Sigma, \delta, s, c)$

$\Sigma = \{0, 1\}$

$Q \times \Sigma \rightarrow Q$



# Building DFA

## Example

Define a DFA which recognizes

$$L_2 = \{w \mid \text{the length of } w \text{ is one more than a multiple of } 3\}$$

A multiple of 3 has remainder 0:

$$3k, \quad k \in \mathbb{N} \rightarrow \{0, 1, 2, \dots\}$$

One more than that

$$3k+1$$

That means:

$$|w| \equiv 1 \pmod{3}$$

$$\underline{3k}$$

0

3

6

9

12

$$\underline{3k+1}$$

1

4

7

10

13

} These lengths are accepted by the machine



# Building DFA

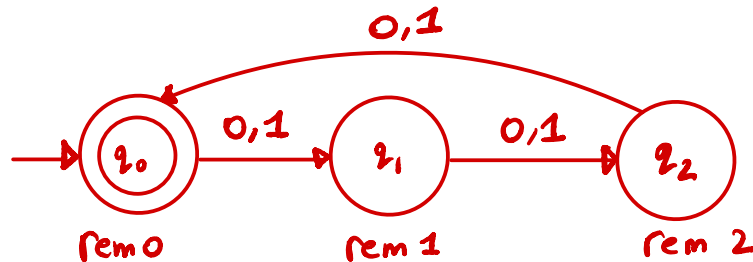
## Example

Define a DFA which recognizes

$L_2 = \{w \mid \text{the length of } w \text{ is one more than a multiple of } 3\}$

$L = \{w \mid \text{the length of } w \text{ is multiple of } 3\}$

|      | $k=0$ | $k=1$ | $k=2$ | $k=3$ |
|------|-------|-------|-------|-------|
| $3k$ | 0     | 3     | 6     | 9     |



$\swarrow = 3$

$$0 \leq r < 3$$

0, 1, 2

$$n = 1$$

$$1 = 3 \times 0 + 1 \quad \text{rem} = 1$$

# Building DFA

## Example

Define a DFA which recognizes

$L_2 = \{w \mid \text{the length of } w \text{ is one more than a multiple of } 3\}$

$L = \{w \mid \text{the length of } w \text{ is multiple of } 3\}$

|        | $k=0$ | $k=1$ | $k=2$ | $k=3$ |
|--------|-------|-------|-------|-------|
| $3k$   | 0     | 3     | 6     | 9     |
| $3k+1$ | 1     | 4     | 7     | 10    |

$$1 \div 3 = 1$$

$$4 \div 3 = 1$$

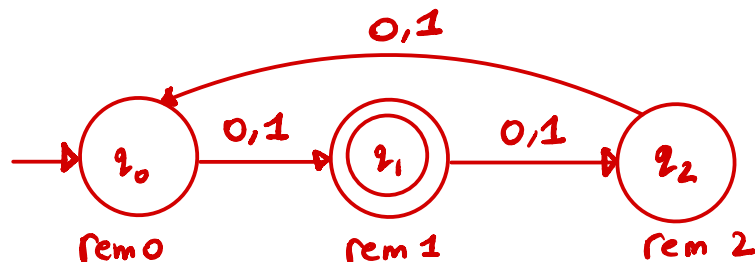
$$7 \div 3 = 1$$

$$0 \leq r < 3$$

$$0, 1, 2$$

$$n=1$$

$$1 = 3 \times 0 + 1 \quad \text{rem} = 1$$



## Formal Statement

For any integer  $n$  and any positive integer  $d$ , there exist unique integers:

$$n = dq + r$$

Such that:

$q$  = quotient

$r$  = remainder

$$0 \leq r < d$$

$$n = dq + r$$

$q$  = result obtained by dividing  $n$  by  $d$ .

| <u><math>n</math></u> | <u>Division by 3</u>                       | <u>Remainder</u> |
|-----------------------|--------------------------------------------|------------------|
| 6                     | $6 = 3 \times 2 + 0$ $\frac{6}{3} = 2 = q$ | 0                |
| 7                     | $7 = 3 \times 2 + 1$ $\frac{7}{3} = 2 = q$ | 1                |
| 8                     | $8 = 3 \times 2 + 2$                       | 2                |
| 9                     | $9 = 3 \times 3 + 0$                       | 0                |
| 10                    | $10 = 3 \times 3 + 1$                      | 1                |

So numbers cycle in remainders:

0, 1, 2, 0, 1, 2, - - -

# Building DFA

$$0 \leq r < 3$$

$$0, 1, 2$$



## Example

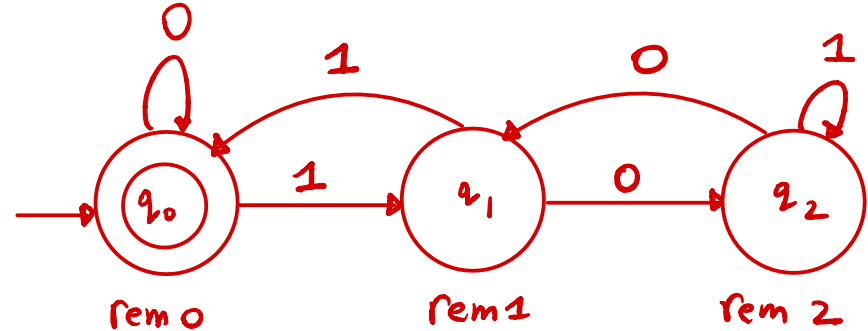
Define a DFA which recognizes

$L_3 = \{w \mid w \text{ when interpreted as a binary number, is divisible by 3}\}$

$$n = 1001 = 9 \div 3 = 0$$

$$(2 \times 0 + 1) \div 3 = 1 \div 3 = 1$$

| Decimal | Binary | remainder |
|---------|--------|-----------|
| 0       | 0      | 0 } ✓     |
| 1       | 1      | 1 }       |
| 2       | 10     | 2 }       |
| 3       | 11     | 0 } ✓     |
| 4       | 100    | 1 }       |
| 5       | 101    | 2 }       |
| 6       | 110    | 0 } ✓     |
| 7       | 111    | 1 }       |



$$\text{New remainder} = (2 \times \text{Old rem} + b) \bmod 3$$

$$(2 \times 1 + 1) \div 3 = 2$$

$$3 \div 3 = 0$$

Think in decimal first

Number  $(7)_{10}$

Append a digit 3 in base-10

734

$$10 \times 7 + 3 = 73$$

$$10n + d$$

Same idea in binary (base 2)

$$(101)_2 = (5)_{10} = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 0 + 1 = 5$$

↓ ↓ ↓  
2 1 0

$$(1010)_2 = \underline{2n + b} = \underline{2 \times 5} + 0 = 10$$

↳ appending 0

Same idea in binary (base 2)

$$(101)_2 = (5)_{10} = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 0 + 1 = 5$$

$\downarrow \downarrow \downarrow$   
2 1 0

$$\underbrace{(101)_2}_5 = 2n + b = 2 \times 5 + 0 = 10$$

$\hookrightarrow$  appending 0

$$\underbrace{(1011)_2}_5 = 2n + b = 2 \times 5 + 1 = 11$$

$\hookrightarrow$  appending 1

Pattern:

If current value is  $n$ :

append 0  $\rightarrow$  new value =  $2n$

append 1  $\rightarrow$  new value =  $2n + 1$

# Building DFA

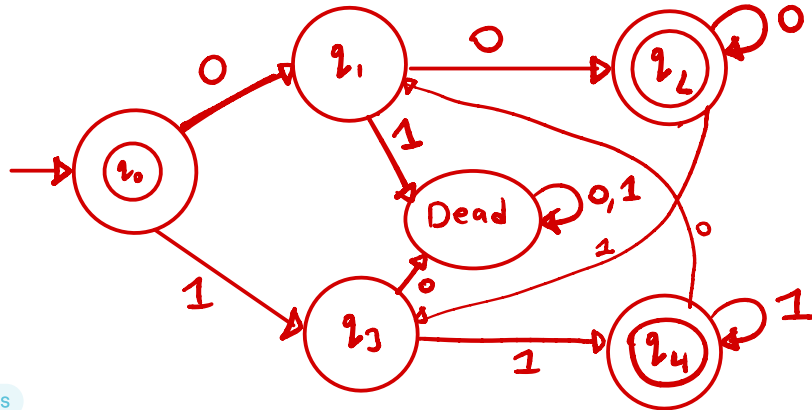
## Example

Define a DFA which recognizes

$L_4 = \{w \mid \text{all characters are repeated at least once wherever they appear in the string}\}$

$L_4 = \{ \epsilon, 0011, \underline{00}, \underline{000111100}, 11, \text{---} \}$   
at least 1  $\geq 1$  at least 1 time

$01 \notin L_4$   
 $00100 \notin L_4$   
 $0101 \notin L_4$



# Building DFA

## Example

Define a DFA which recognizes

~~$L_5$~~  = {w | w contains an equal number of occurrences of the substrings 01 and 10}

~~$L_5$~~  = {w starts and end with 0 or w start and ends with 1}

$L_5 = \{ \underbrace{01}_0 \overbrace{10}^1, \overbrace{0110}^2, 1001 \dots \}$

01=10

011  $\notin L_5$   
100  $\notin L_5$

01011010



# Building DFA



## Example

Define a DFA which recognizes

$L_6 = \{w \mid 01 \text{ appears even number of times in } w \text{ as a substring}\}$

# Building DFA

## Example

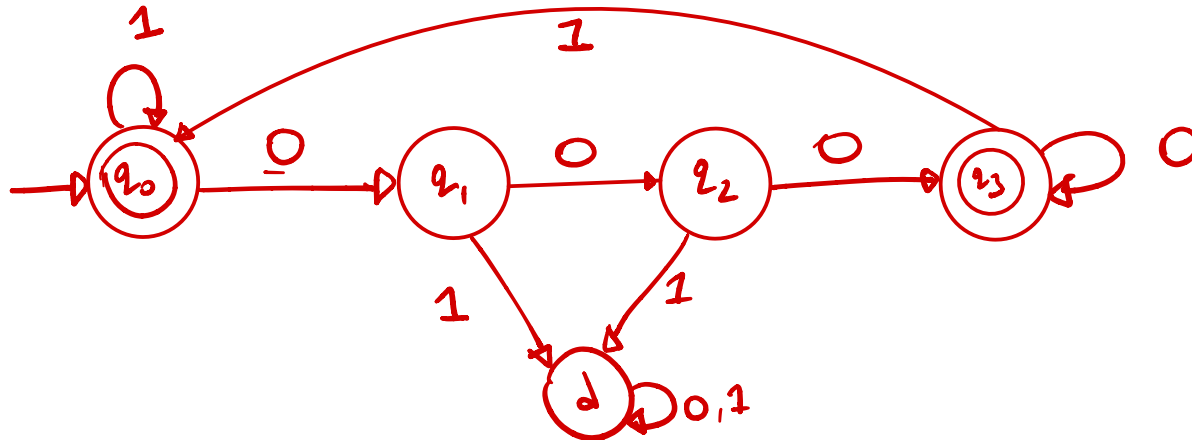
Define a DFA which recognizes

$L_7 = \{w \mid \text{all strings in which every run of } \underline{0}\text{s has } \underline{\text{length at least } 3}\}$

$L_7 = \{ \underline{000}, 111, 1, \underline{\epsilon}, 00001, 1100001000, \dots \}$

0100  $\notin L_7$   
0010  $\notin L_7$

$\Sigma = \{0, 1\}$



# Building DFA



## Example

Define a DFA which recognizes

$L_8 = \{w \mid \text{all strings } w \text{ such that in every prefix of } w, \text{ the number of 0s and 1s differ by at most } 1\}$

# Building DFA



## Example

Define a DFA which recognizes

$L_9 = \{w \mid \text{all strings containing at least two 0s and at least one 1.}\}$



# Building DFA



## Remember

States are our only (computer) memory.

Design and pick states with specific roles / tasks in mind.

*"Have not seen any of desired pattern yet"*

*"Trap state"*